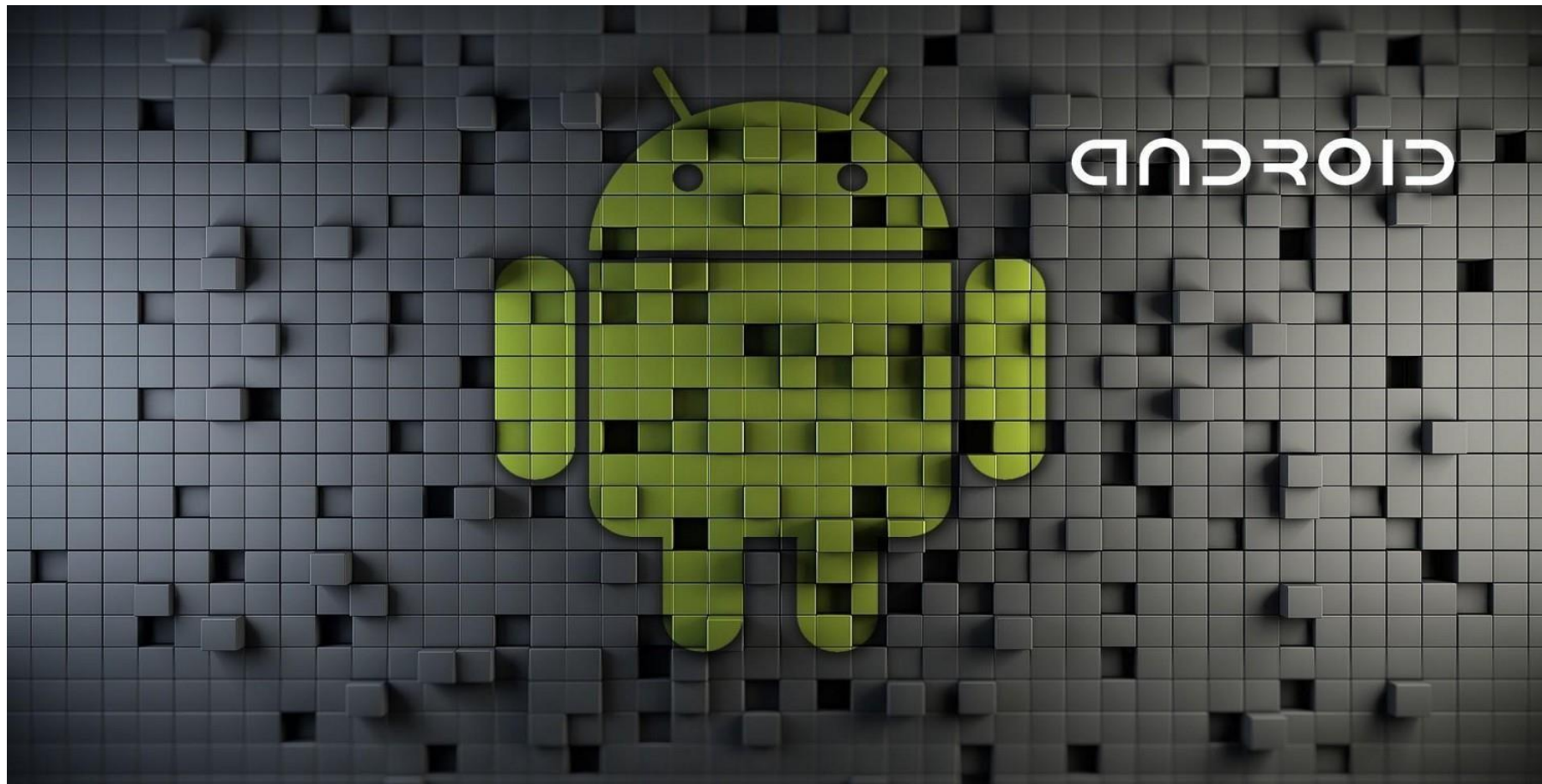


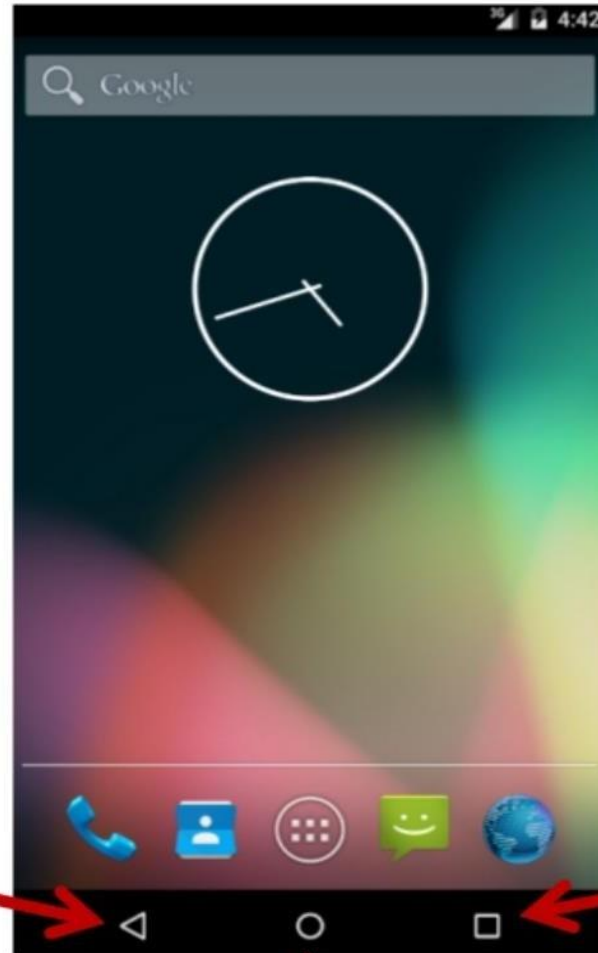


ANDROID ACTIVITY



android

Android interface



Back button

Move backward through the history of screens users previously visited



Home button

Visualize the Android home screen

Recent button

Jump instantly from one task to another by navigating a list of recently opened apps

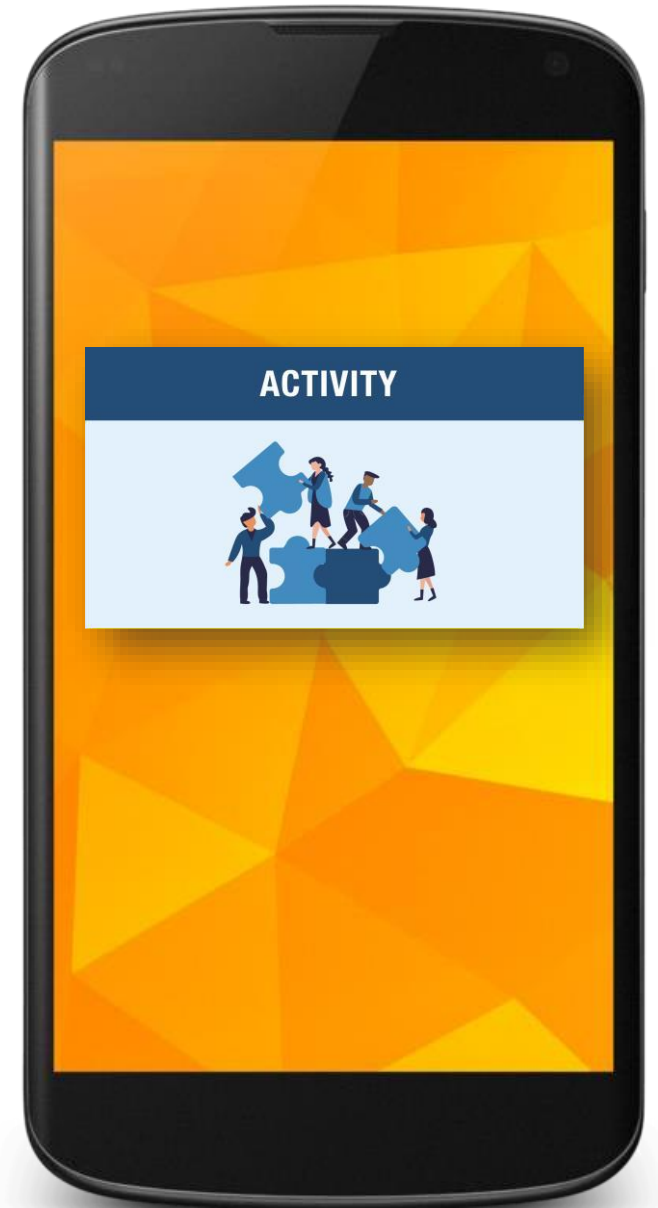




android

ACTIVITY

- An activity is a single screen of an application that lets us see and interact to perform an activity. Usually, an application contains many screens and each screen extends Activity() class.
- Activity is nothing but a java class in Android which has some pre-defined functions which are triggered at different App states, which we can override to perform anything we want.
- Activity class provides us with empty functions allowing us to be the controller of everything.



WHAT IS ACTIVITY?

- When we work on an application what we see is a UI on the screen which is an activity. Most of the applications that we use have many activities. Among all those activities, one is the MainActivity() & the rest are its ChildActivities(). Generally, the first page that appears on the screen when an application opens is referred to as MainActivity(). This main activity interacts with the child activities and lets the user access them.
- Activities are stored in a stack of Activities, wherein the current activity holds the highest position.





android

WHAT IS ACTIVITY?

- Activity is just a single screen. Yes it is that simple. So in our case - Main Menu Screen is one activity, Game Screen is another activity and Score Screen is another activity. Activity is a screen that displays some information to the user or requires some input from the user. Activity handles the interaction with the user.





android

WHAT IS ACTIVITY?

- **Android Activity:** is the main component of Android applications. The activity gives user interaction, you can set UI `setContentView(View)` in activity to show the user interface. Like when you open mail apps (Gmail etc) you will see the list of emails. That is showing in a list on activity or using other components.





android

ACTIVITY EXAMPLE (face book)



Activity 1: Login



Activity 2:
NewsFeed



Activity 3:
Profile



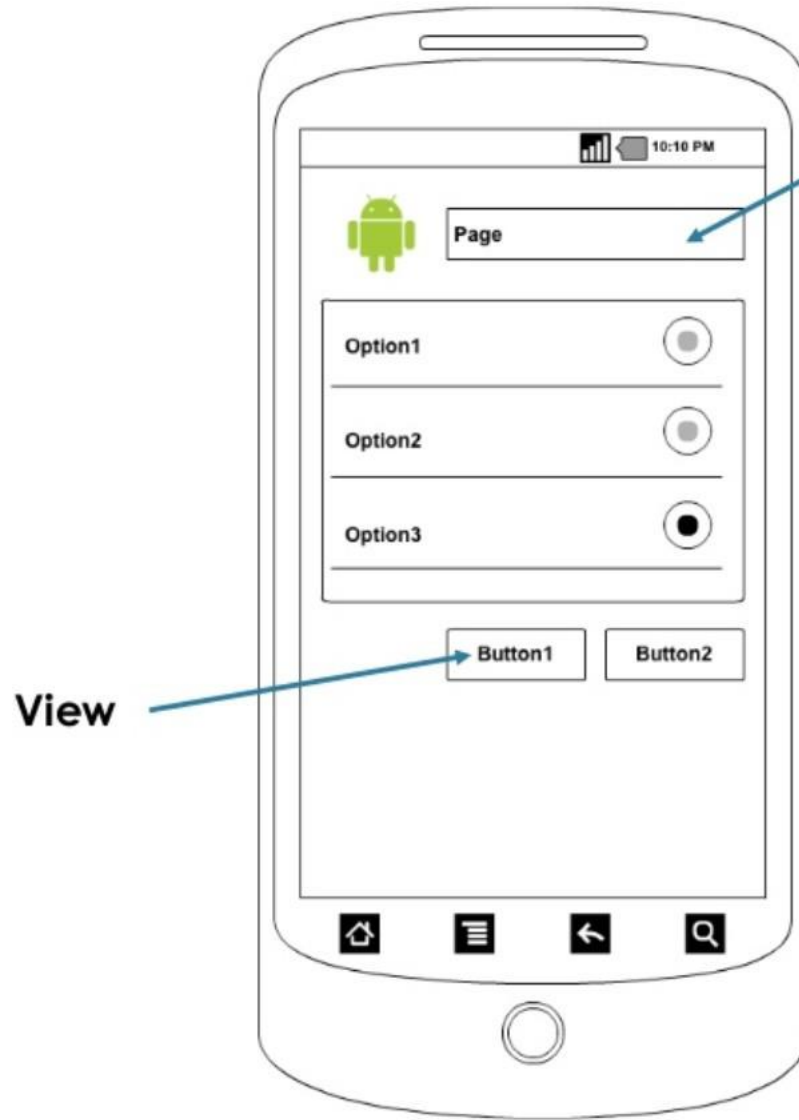
Activity 4:
Notifications

ACTIVITIES AND VIEWS



android

- The **activity** is a single, focused thing that the user can do
 - Each activity is associated with a window in which to draw the user interface
- The **view** is the basic building block for user interface components
 - Responsible for drawing and event handling
 - Examples: button, textbox



View

The **Activity** is the task that can be performed by interacting with the user interface

Application building
blocks: UI components





THE MANIFEST FILE

- Every app project must have an **AndroidManifest.xml** file (with precisely that name) at the root of the project source set. The manifest file describes essential information about your app to the Android build tools, the Android operating system, and Google Play.

ACTIVITES DECLARATION IN MANIFEST FILE



android

The screenshot shows the AndroidManifest.xml file in Android Studio. The file contains the following XML code:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.summitapps">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="SummitApps"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/Theme.SummitApps">

        <activity android:name=".MainActivity2"></activity>
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Red boxes highlight the activity declarations: `<activity android:name=".MainActivity2"></activity>` and `<activity android:name=".MainActivity">`. Red arrows point from the Resource Manager to the corresponding layout files: `activity_main.xml` and `activity_main2.xml`.



1- PACKAGE NAME & APPLICATION ID

- The app's package name, which usually matches your code's namespace. The Android build tools use this to determine the location of code entities when building your project. When packaging the app, the build tools replace this value with the application ID from the Gradle build files, which is used as the unique app identifier on the system and on Google Play



2- APP COMPONENTS

- The components of the app, which include all activities, services, broadcast receivers, and content providers. Each component must define basic properties such as the name of its Kotlin or Java class. It can also declare capabilities such as which device configurations it can handle, and intent filters that describe how the component can be started.
 - `<activity>` for each subclass of `Activity`.
 - `<service>` for each subclass of `Service`.
 - `<receiver>` for each subclass of `BroadcastReceiver`.
 - `<provider>` for each subclass of `ContentProvider`.



2- INTENT FILTERS

- Intent Filters are expressions in the Manifest file of an Android Application. These are responsible for deciding the behavior of the Intent. Intent filters that we mention in the Manifest file can be nested with the application components.
- Basically, Intent Filters can define the behavior of Intents using three Elements, that are-
 - ✓ **<actions>** – Action name defines the intent action that it'll accept.
 - ✓ **<data>** – Data defines the data that is acceptable.
 - ✓ **<category>** – Category defines the name of the Intent Category that is acceptable.

When there are multiple activities set as main and launcher with an intent filter in the manifest. Then first activity considers as Launcher activity, and android launch or open the first activity.

```
<category android:name="android.intent.category.LAUNCHER" />
```



4- PERMISSIONS

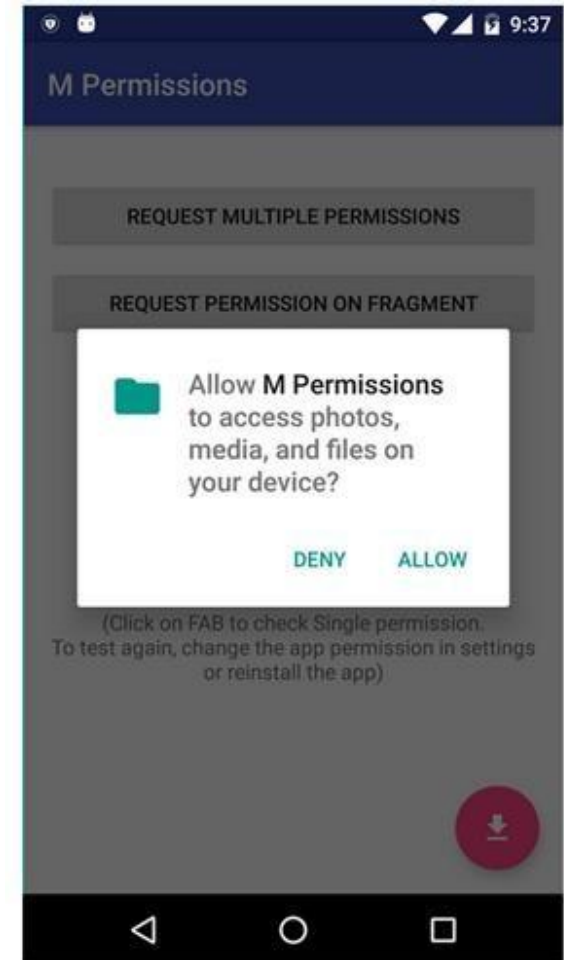
- Android apps must request permission to access sensitive user data (such as contacts and SMS) or certain system features (such as the camera and internet access). Each permission is identified by a unique label. For example, an app that needs to send SMS messages must have the following line in the manifest:

```
<manifest ... >
```

```
    <uses-permission  
        android:name="android.permission.SEND_SMS"/>
```

```
    ...
```

```
</manifest>
```





5- DEVICE COMPATABILITY

- The manifest file is also where you can declare what types of hardware or software features your app requires, and thus, which types of devices your app is compatible with. Google Play Store does not allow your app to be installed on devices that don't provide the features or system version that your app requires.



- There are several manifest tags that define which devices your app is compatible with. The following are just a couple of the most common tags.

1 - <uses-feature>

- The <uses-feature> element allows you to declare hardware and software features your app needs. For example, if your app cannot achieve basic functionality on a device without a compass sensor, you can declare the compass sensor as required with the following manifest tag:

```
<manifest ... >
    <uses-feature android:name="android.hardware.sensor.compass"
                  android:required="true" />
    ...
</manifest>
```



2- <uses-sdk>

- Each successive platform version often adds new APIs not available in the previous version. To indicate the minimum version with which your app is compatible, your manifest must include the <uses-sdk> tag and its minSdkVersion attribute.
- However, beware that attributes in the <uses-sdk> element are overridden by corresponding properties in the build.gradle file. So if you're using Android Studio, you must specify the minSdkVersion and targetSdkVersion values there instead:

```
android {  
    defaultConfig {  
        applicationId 'com.example.myapp'  
  
        // Defines the minimum API level required to run the app.  
        minSdkVersion 15  
  
        // Specifies the API level used to test the app.  
        targetSdkVersion 28  
  
        ...  
    }  
}
```



android

Tags in Manifest xml File

- `<action>`
- `<activity>`
- `<activity-alias>`
- `<application>`
- `<category>`
- `<data>`
- `<grant-uri-permission>`
- `<instrumentation>`
- `<intent-filter>`
- `<manifest>`
- `<meta-data>`
- `<permission>`
- `<permission-group>`
- `<permission-tree>`
- `<provider>`
- `<receiver>`
- `<service>`
- `<uses-configuration>`
- `<uses-library>`
- `<uses-permission>`
- `<uses-sdk>`

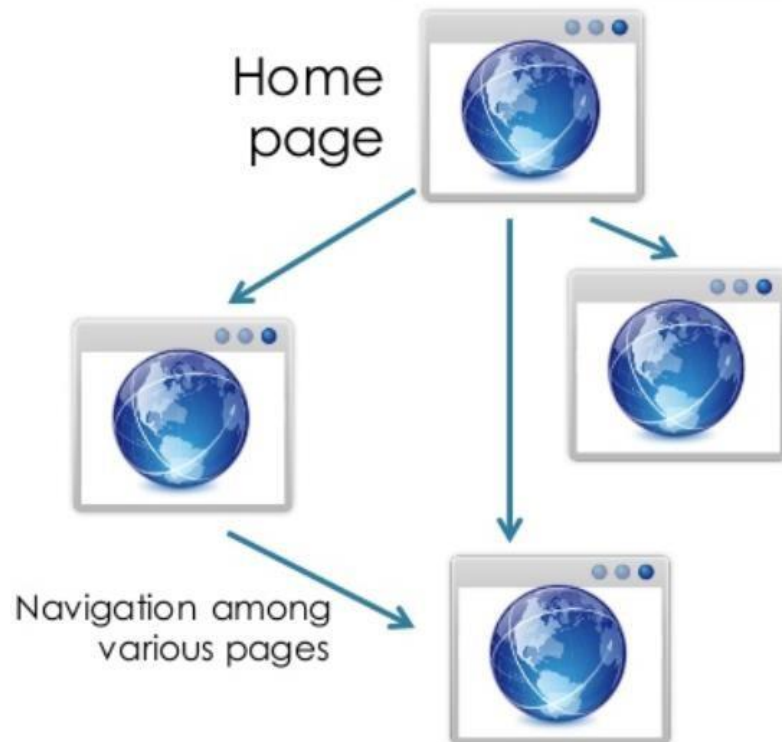
Analogy: web pages and activities



android

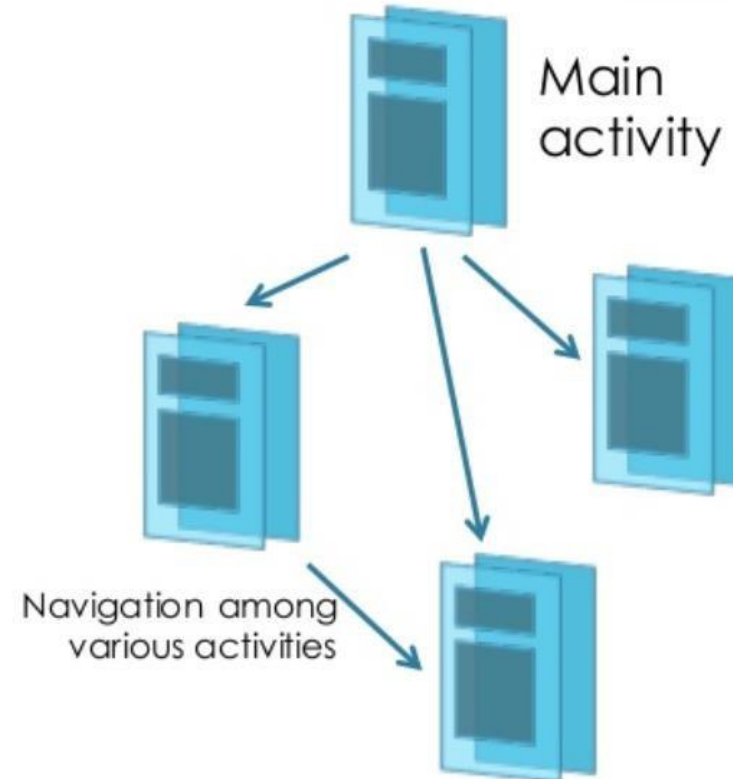
Home page: Index.html

Website



Home page: MainActivity

Application





Screen

activity



1-

 activity_main.xml ×

2-

 MainActivity.java ×



android

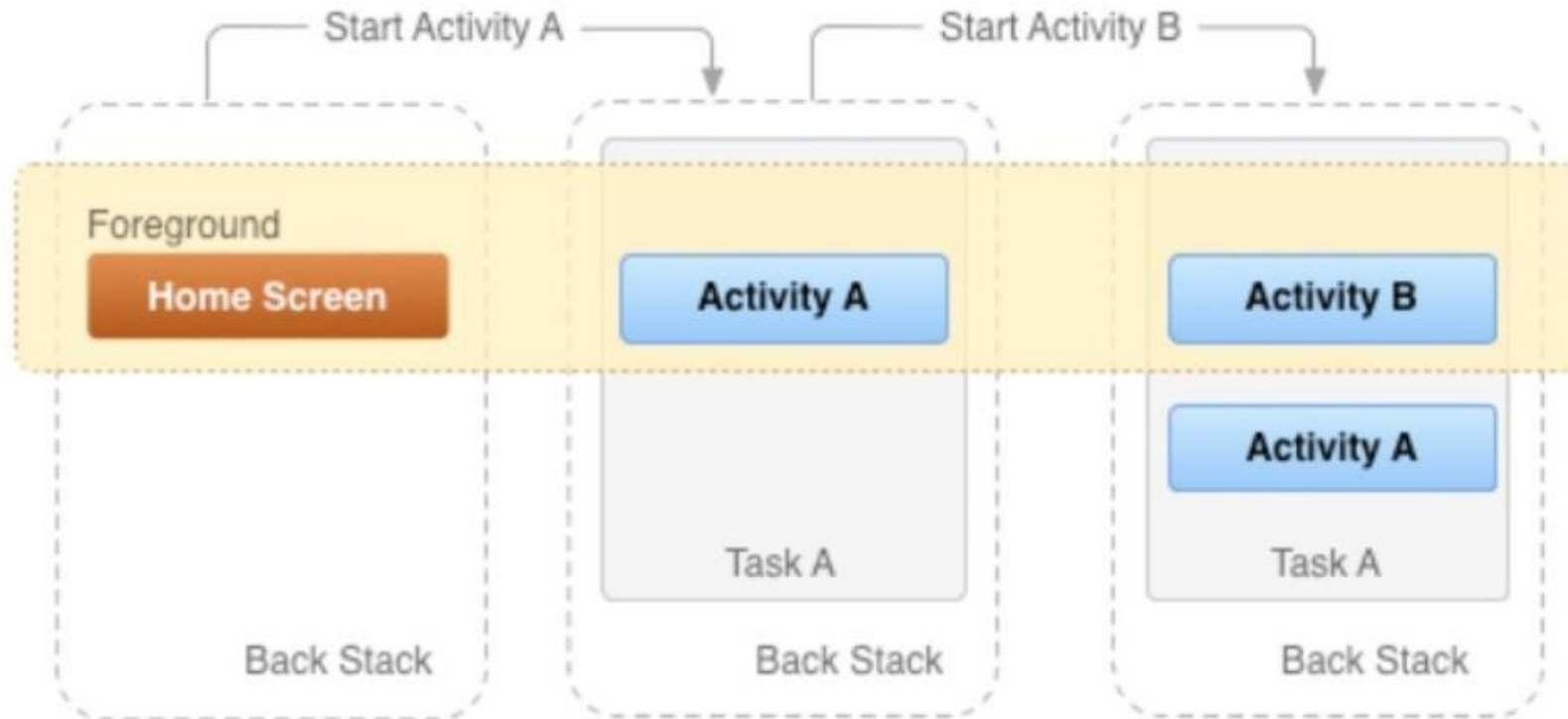
ACTIVITY STACK

- Activities in the system are managed as an activity stack.
- When a new activity is started, it is placed on the top of the stack and becomes the running activity
 - > The previous activity always remains below it in the stack, and will not come to the foreground again until the new activity exits.



android

ACTIVITY STACK



Activity A and Activity B are launched in Task A.
By default all activities in the same application belong to a single task.